

Progetto Algoritmi e Strutture Dati

Analisi comparativa di algoritmi di ordinamento

Federico Del Pup
Luigi Pascu
Matteo Passador
Stefano Toneguzzo

Maggio 2025

Indice

1. Introduzione e scopo del progetto
2. Organizzazione del progetto
3. Analisi dei grafici nei casi intermedi
4. Analisi dei grafici nei casi peggiori
5. Partecipanti al progetto

1 Introduzione e scopo del progetto

Lo scopo del progetto di laboratorio è progettare, implementare e analizzare algoritmi di ordinamento. In particolare, il progetto richiede:

- l'implementazione corretta di quattro algoritmi di ordinamento per array di interi, tre fissati dal docente (Quicksort, Countingsort e Quicksort 3-way) e uno scelto dal gruppo di lavoro (Mergesort)
- la misurazione sperimentale dei tempi medi di esecuzione al variare della dimensione dell'array n e del range m degli interi che compongono l'array
- la produzione di grafici comparativi che mostrino le prestazioni degli algoritmi sul tempo t di esecuzione in base ai due parametri scelti
- la stesura di una relazione tecnica in cui si discutono le scelte implementative e si analizza la complessità empirica osservata

L'obiettivo finale è valutare correttamente le prestazioni degli algoritmi, notando caratteristiche positive e difetti di ognuno

2 Organizzazione del progetto

Scelta del linguaggio

Il motivo dietro alla scelta di Python risiede in diverse caratteristiche favorevoli del linguaggio: essendo Python un **linguaggio ad alto livello** ha permesso di sviluppare rapidamente il codice senza compromettere la chiarezza e la gestione. Inoltre Python offre una vasta gamma di librerie e strumenti integrati utili per il progetto. In particolare, la libreria *time* ha permesso di effettuare misurazioni precise dei tempi di esecuzione utilizzando la funzione *perf_counter()*, mentre librerie come *matplotlib* e *numpy* hanno facilitato la visualizzazione e l'analisi dei risultati ottenuti attraverso **grafici comparativi**.

Scelta del quarto algoritmo

Come quarto algoritmo da analizzare, è stato scelto Mergesort per la sua efficienza garantita in tutti i casi, con complessità $\Theta(n \log(n))$ e la stabilità dell'ordinamento, che lo rendono un buon punto di confronto con gli altri algoritmi richiesti.

Strumenti e ambiente di sviluppo

Al fine di coordinare meglio lo sviluppo del progetto, è stato creato un file condiviso su **Google Colab**. Questo ambiente collaborativo ha permesso a tutti i membri del gruppo di lavorare simultaneamente sul codice, apportare modifiche in tempo reale ed eseguire test sugli algoritmi dedicati alla generazione dei grafici. Il codice è stato inizialmente sviluppato in **moduli separati**, ciascuno dedicato a una specifica funzionalità. Successivamente, i vari componenti sono stati integrati in un'unica soluzione coerente e funzionante. Per la generazione dei grafici abbiamo utilizzato un *Acer Predator Helios NEO 16*, con **CPU Intel i9 14900HX** e **RAM 16GB**, utilizzando l'ambiente di sviluppo **Visual Studio Code**. Infine la relazione è stata scritta in LaTeX su **Overleaf**.

3 Analisi dei grafici nei casi intermedi

I campioni

Per realizzare i grafici, è stato suddiviso l'intervallo preso in esame in 100 campioni. Piuttosto che usare una distribuzione uniforme, è stata scelta una **serie geometrica** che parte dal valore minimo e cresce gradualmente fino al massimo. Questo approccio permette di avere una **maggiore densità** di punti nella parte bassa del range, dove le differenze tra gli algoritmi sono spesso più significative. Per ciascuno di questi 100 campioni, è stato condotto un test: sono state eseguite 1000 prove indipendenti di ogni algoritmo. In ogni prova è stato generato un nuovo array con valori casuali. Per ogni algoritmo, si è misurato il tempo di esecuzione al netto di quello per inizializzare gli array. Poi è stata calcolata

la media dei tempi ottenuti. Questa metodologia ha permesso di ottenere dati affidabili e significativi, riducendo l'influenza di fattori casuali e centrando sulle effettive prestazioni degli algoritmi di ordinamento

Dimensione variabile

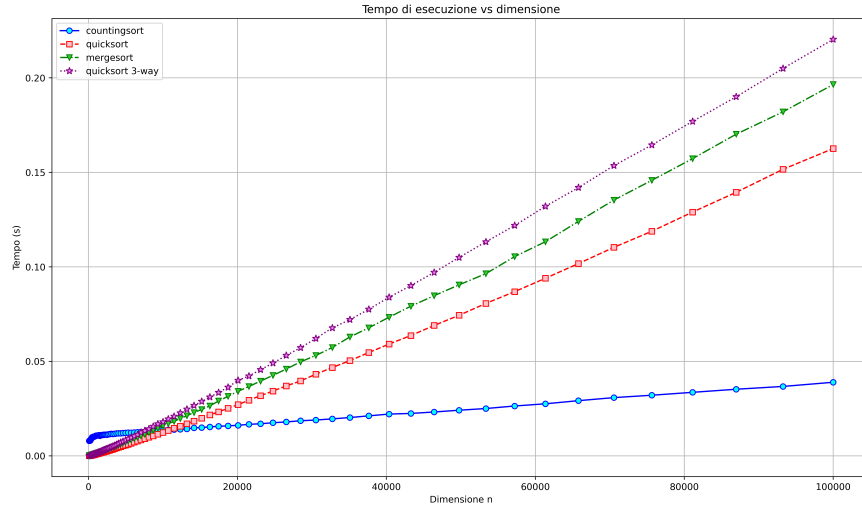


Figure 1: Grafico in funzione della dimensione dell'array n

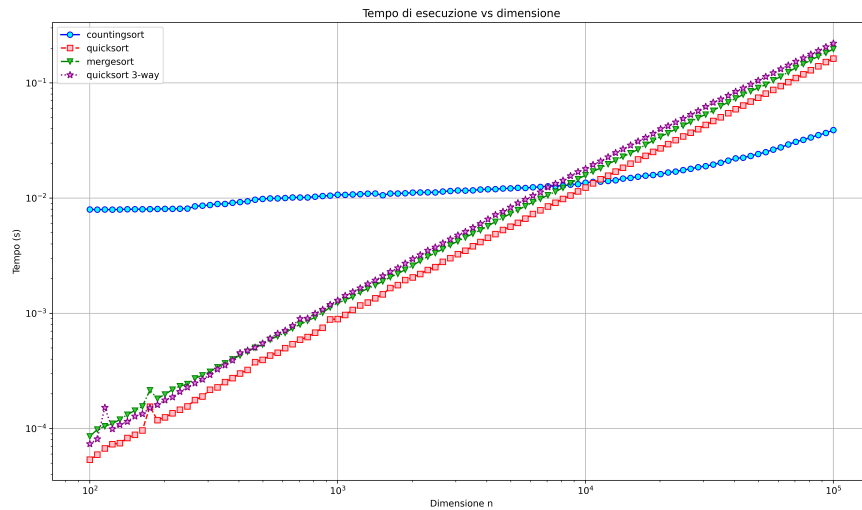


Figure 2: Grafico in scala doppia logaritmica in funzione della dimensione dell'array n

Dal grafico [Fig. 1] si nota che l'asse delle ascisse mostra una dimensione n crescente, mentre il range di interi è fisso a 10^5 elementi. Sull'asse delle ordinate è riportato il tempo t di esecuzione degli algoritmi per l'ordinamento dell'array.

Si nota che il Countingsort risulta **più lento** degli altri algoritmi quando lavoriamo con array di ordini di grandezza inferiori. All'aumentare della dimensione dell'array il Countingsort diventa notevolmente più veloce (dall'ordine di grandezza $n = 10^4$ in poi)

Questo cambiamento si spiega perché, essendo l'array costituito da interi presi da un range fisso di valori, all'aumentare della dimensione n cresce anche il numero di elementi **ripetuti**. In questa circostanza il Countingsort eccelle grazie alla sua struttura. Inoltre la sua complessità computazionale è inferiore agli altri algoritmi, $O(n+k)$. Essendo $k \approx m$ fissato a priori, all'aumentare della dimensione la complessità degli altri tre algoritmi, $O(n \log(n))$, rende i tempi di esecuzione meno vantaggiosi rispetto a quelli di Countingsort.

Gli altri tre algoritmi confrontati (Mergesort, Quicksort e Quicksort 3-way) mostrano invece prestazioni simili tra loro, dato che condividono la **stessa complessità computazionale** media di $O(n \log(n))$.

Nel grafico in scala doppia logaritmica [Fig. 2] si nota molto meglio la differenza prestazionale al variare dell'ordine di grandezza della dimensione dell'array n

- per $n \leq 10^4$: Il Countingsort è molto più inefficiente degli altri algoritmi
- per $n > 10^4$: Il Countingsort diventa il più efficiente

La scala doppia logaritmica ci consente di valutare per ogni **ordine di grandezza** della dimensione n qual è l'algoritmo più efficiente.

Range variabile

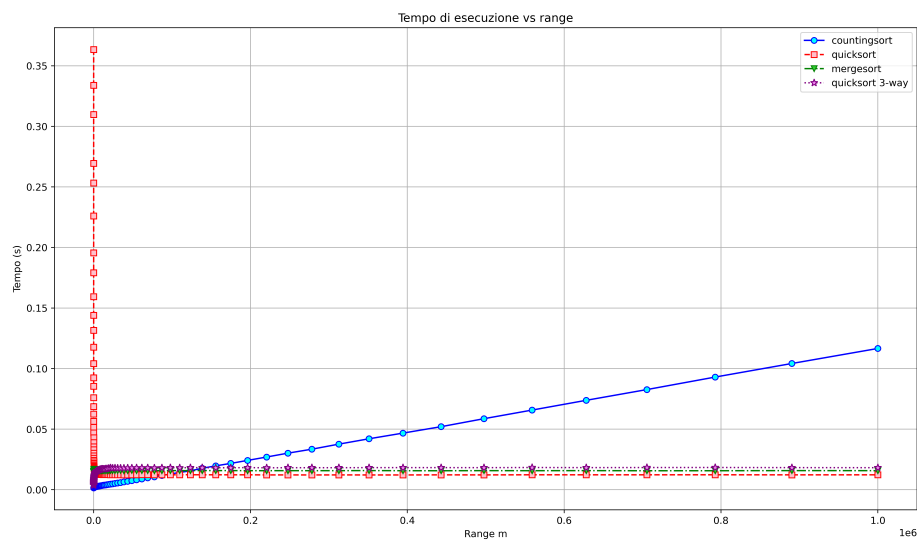


Figure 3: Grafico in funzione del range di interi m

Il grafico [Fig. 3] mostra i tempi di esecuzione degli algoritmi di ordinamento al variare del range m di interi che costituiscono gli array. Gli array campione hanno dimensione n fissata a 10^5 elementi.

Quando il range di interi diventa più ampio, il Countingsort risulta l'algoritmo **meno efficiente**, poiché deve memorizzare un **numero maggiore di valori**. Dal grafico lineare si osserva che: con range piccoli, il Quicksort classico è il più lento mentre il Countingsort è il più veloce

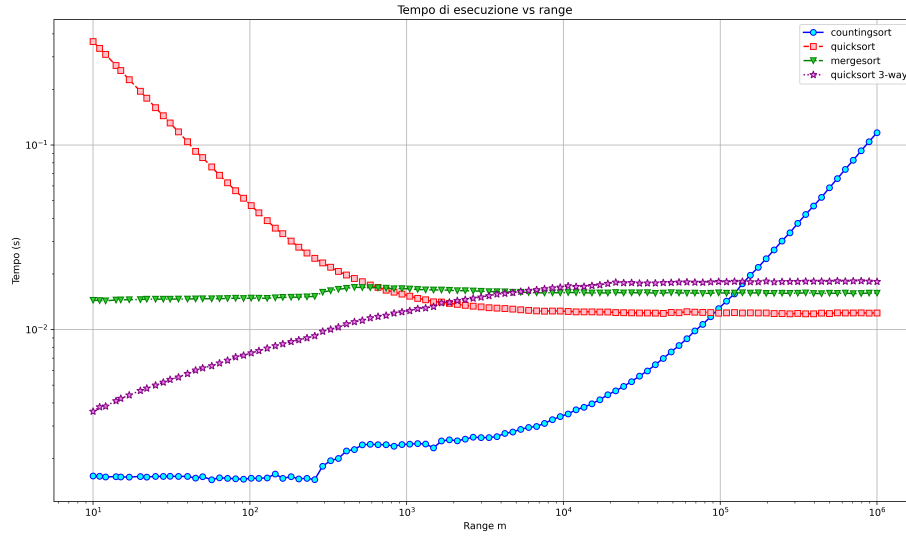


Figure 4: Grafico in scala doppia logaritmica in funzione del range di interi m

Nel grafico [Fig.4] in scala doppiamente logaritmica, l'andamento risulta più chiaro: Con il range $m < 10^3$ il Quicksort classico è il meno efficiente, poiché la presenza di **molte valori ripetuti**, tipica di range ristretti, ne penalizza le prestazioni. Il Quicksort 3-way è invece ottimale in confronto, grazie alla sua capacità di gestire efficacemente i valori ripetuti, problema che si annulla all'aumento del range valutato.

Allo stesso modo il Countingsort si comporta meglio con range piccoli, con tempi di esecuzione minori degli altri algoritmi. Ciò accade perché Countingsort non effettua confronti tra gli elementi dell'array, ma ne conta l'**occorrenza** di ogni valore.

Quando l'ordine di grandezza supera 10^3 i tempi di esecuzione delle due versioni di Quicksort e di Mergesort diventano stabili, in quanto il numero di elementi ripetuti in un'array risulta **trascurabile**.

Con il range $m > 10^5$ il Countingsort diventa il più lento, poiché la crescita del range aumenta il costo di memorizzazione. Gli altri algoritmi (Mergesort, Quicksort, Quicksort 3-way) mostrano tempi di esecuzione simili.

Il Mergesort mantiene un tempo di esecuzione **costante** all'aumentare del range m . Questo algoritmo non è influenzato dalla presenza di valori ripetuti in quanto non fa uso di *pivot* e non crea strutture legate al valore massimo e minimo presenti nell'array. Lavora solo in funzione della dimensione dell'array.

4 Analisi dei grafici nei casi peggiori

Generazione di caso peggiore

Per analizzare i casi peggiori è stato generato un array con valori unici inseriti in ordine decrescente. Per il Countingsort si è optato per fissare il valore massimo e il valore minimo in modo da ottenere un array col massimo range possibile. Questa scelta deriva dal fatto che il caso peggiore del Countingsort si verifica quando l'array contiene elementi tutti diversi e distribuiti su un ampio intervallo di valori: in tal caso, l'algoritmo è costretto ad allocare molta memoria per gestire ciascun valore singolarmente. Per quanto riguarda Mergesort, abbiamo generato un array con un metodo chiamato **alternanza perfetta**, in cui la sequenza degli elementi è generata in modo che ogni metà abbia elementi intercalati, ovvero che si alternino elementi grandi e piccoli e l'algoritmo è costretto a confrontare ogni singolo elemento

Grafici

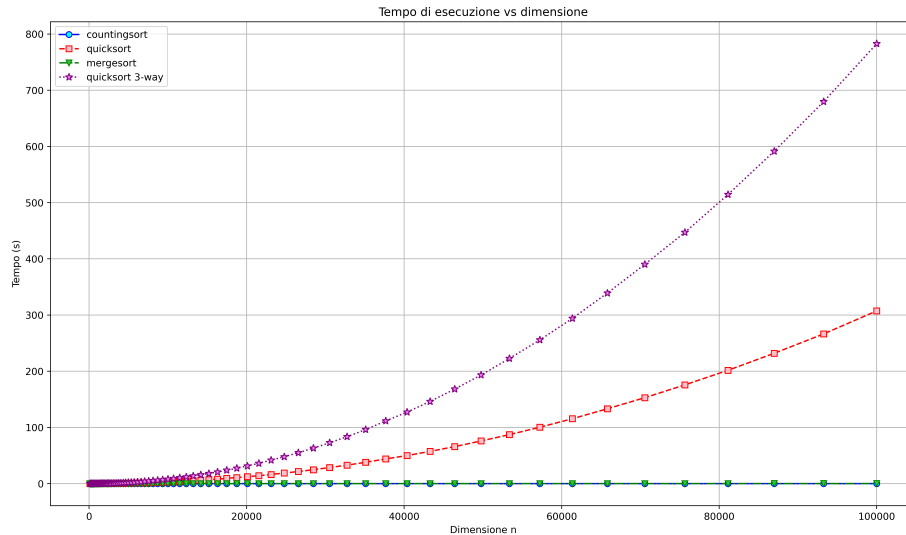


Figure 5: Grafico in scala lineare dei casi peggiori in funzione della dimensione n

Dal grafico lineare [Fig. 5] si nota che gli algoritmi più lenti sono il Quicksort e il Quicksort 3-way, in quanto presentano una complessità nel caso peggiore pari a $O(n^2)$. Il Quicksort 3-way, eseguendo un numero maggiore di confronti, risulta essere molto più lento su array di grandi dimensioni, specialmente quando non sono presenti elementi duplicati e gli elementi sono ordinati in modo decrescente.

Per quanto riguarda il Countingsort e il Mergesort, dal grafico lineare non si può fare una valutazione dettagliata perché le loro curve sono sovrapposte e risultano comunque molto più veloci rispetto agli altri due algoritmi.

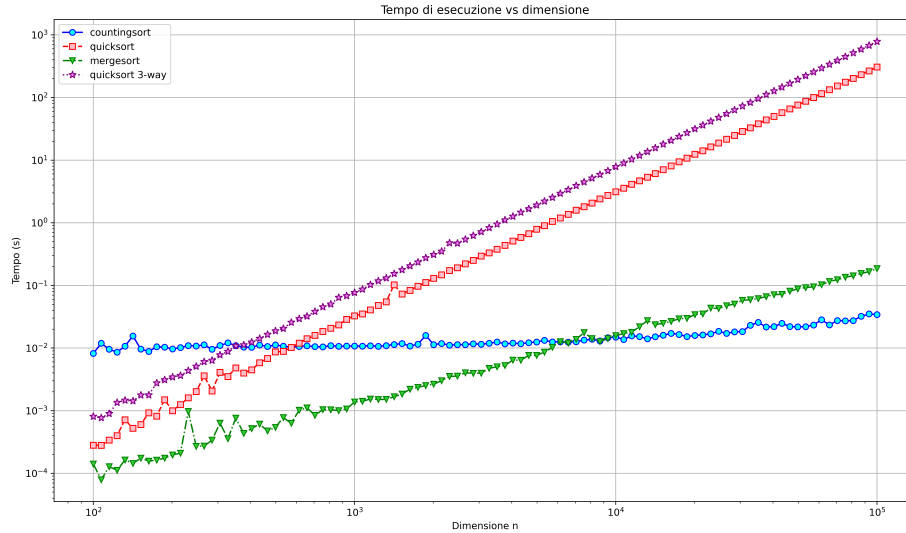


Figure 6: Grafico in scala doppia logaritmica dei casi peggiori in funzione della dimensione n

Per analizzare meglio i comportamenti, si passa alla scala doppiamente logaritmica [Fig. 6], dalla quale si osserva che il CountingSort cresce molto lentamente al crescere della dimensione, ma nelle fasi iniziali, su array di piccole dimensioni, risulta essere l'algoritmo meno efficiente.

Notiamo che due algoritmi non cambiano comportamento:

- Il Mergesort, avendo complessità $O(n \log(n))$, mostra una crescita moderata. Confrontandolo con i suoi casi medi [Fig. 2] ottiene le **stesse prestazioni**. Ciò accade perché Mergesort divide l'array nello stesso modo in sottovettori indipendenti dai valori che lo compongono.
- Il Countingsort non cambia comportamento rispetto al caso medio [Fig. 2]. Questo si può spiegare in quanto nel caso medio sono stati generati array con un range molto ampio. A causa di ciò nel caso medio è ugualmente difficile trovare tanti valori ripetuti. Il caso peggiore di Countingsort si verifica quando il range m di interi è molto più grande della dimensione dell'array da ordinare. **Non è sensibile** alla disposizione iniziale dell'array

5 Partecipanti al progetto

Federico Del Pup
Matteo Passador
Luigi Pascu
Stefano Toneguzzo